

从"读文档"到"管理金融判断"

金融场景下的 Agent 系统设计与长文本管理

作者：山鬼



作者：山鬼

金融行业的独特性

今天分享的主题是从"读文档"到"管理金融判断"。在进入技术之前，我想先聊一下金融行业的独特性，agent的价值目前对于行业来说，展现出的两个比较大的价值，一个是业务探索、另一个是场景落地，这些都会要了解这个行业是有哪些问题是落地中需要被解决的。这些问题决定了通用的Agent 方案为什么在金融场景里会失效。

什么是金融长文本？

在我看来金融行业也是信息支撑的行业，所以都会面临长文本的问题，不过不同的是，金融领域的长文本有着一些独特的难点，这个主要是由于行业历史信息结构导致的。

第一：证据分散，且有依赖顺序

比如要回答"这个客户能不能再批一笔500万的短期贷款"，需要的信息来自账务系统的余额、风控系统的还款记录、外部征信报告、内部授信政策条款。没有一份单独的材料能回答这个问题。而且这些证据之间有逻辑顺序——不能先看政策再看征信，因为客户的征信状态会影响哪条政策适用。这是结构性的问题。

第二：切片断链，例外条款无法被语义检索命中

比如政策第3页："单笔短期贷款上限1000万。" 政策第17页："但制造业客户近6个月出现逾期的，新增授信须降级审批。"这两条规则在业务逻辑上是强依赖的。但按512字切片之后，"单笔上限1000万"和"制造业客户逾期降级审批"在向量空间里的相似度并不高。RAG检索到了第3页，没有检索到第17页。信息链断开是一个较为常见的长文本中的问题。

第三：例外条款的权重和主条款一样，但位置是隐蔽的

有些文档有一个特点：正文写上限，另条写降级；产品说明书写收益，风险揭示书写亏损情形；主条款写赔付，附录写例外。这些配对内容在文档结构上是分离的，但在业务逻辑上是不可分割的。如果仅仅召回单个信息，最终的结论大概率会获得一个看起来逻辑正确的错误答案。

第四：数据时效性与信息冲突

金融数据有时效性。比如征信报告是上个月拉的，但客户昨天刚发生了一笔逾期。政策文档有新旧两个版本，检索可能同时返回，而且两个版本对同一情形的规定不同。更复杂的情况是信息冲突：客户在系统内登记的行业分类是"制造业"，工商信息显示的是"贸易业"。两个数据来源给出了不同的输入，而哪个适用于当前判断，需要有人来确认。通用场景里，信息冲突可能是噪音。金融场景里，信息冲突是一个需要明确处理、不能自动决策的问题。

第五：口径不一致

同一个指标，在不同产品、不同年份、不同主体下含义可能不同。"营业收入"在并表口径和母公司口径下数值可能差出数倍。"等待期"在不同险种里的起算方式不同。"逾期"在不同信贷产品里的认定标准不同。Agent必须知道不同来源的数据能不能放在一起比较，不能把不同口径的数字直接拼在一张对比表里。

第六：监管留痕与责任归属

金融场景有一个通用场景里几乎没有的要求：事后可以说清楚这个结论是怎么得出来的。审计要求、监管检查、客户投诉，都可能需要还原推理过程。"模型说可以所以我批了"在金融行业不是一个可以接受的解释。这个要求直接影响系统设计：中间推理步骤不能随便压缩，证据来源必须保留原文和页码，每一个影响最终结论的判断步骤必须可以被追溯。

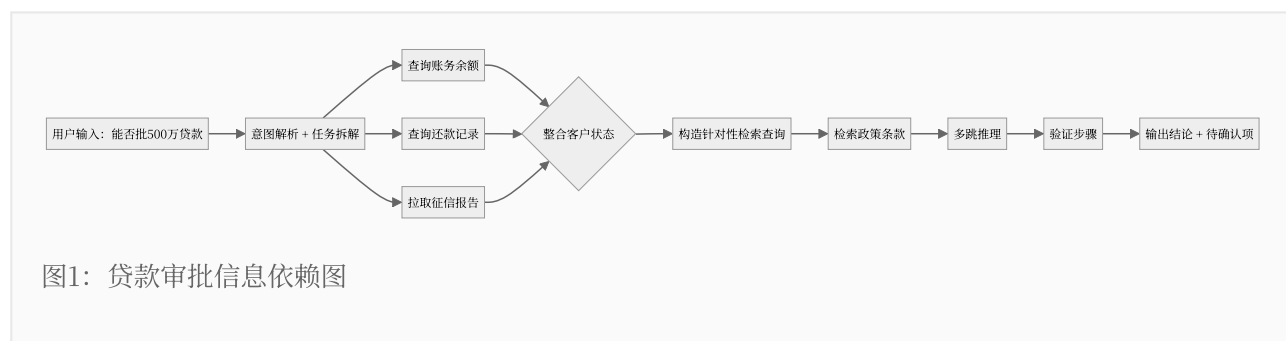
从业务场景到 Agent 系统设计

我们来看一下场景中的agent是如何运作的。当客户经理问"这个客户能不能再批一笔500万的短期贷款"。这个问题看起来是一个查询，但真正开始想怎么让 Agent 回答它的时候，会遇到四个设计上的问题需要决策。

Agent 需要去哪里取信息，顺序是什么？

要回答这个问题，Agent 需要：账务余额、还款记录、征信报告、政策条款。最直接的做法是按顺序调用不同的数据接口。但很快会发现这里的问题：政策条款的检索需要知道客户是什么行业、有没有逾期，而这两条信息要从前面的工具调用里拿。如果顺序写死，就可能在信息还没拿到的情况下去做检索，检索的查询词是不完整的。

所以系统的第一个设计问题是：任务之间有依赖关系，不能顺序执行，需要先建依赖图。



这张图里有一个关键节点：F，整合客户状态。在这个节点之前，三个工具调用可以并行；这个节点之后，检索才正式开始。检索的查询词也不是从用户原始问题直接生成的，而是会从已经确认的客户状态结合初始信息合成的。这就是为什么这道题的检索要做多轮——第一轮用原始问题找主条款，后续几轮用客户的具体情况找例外和降级规则。

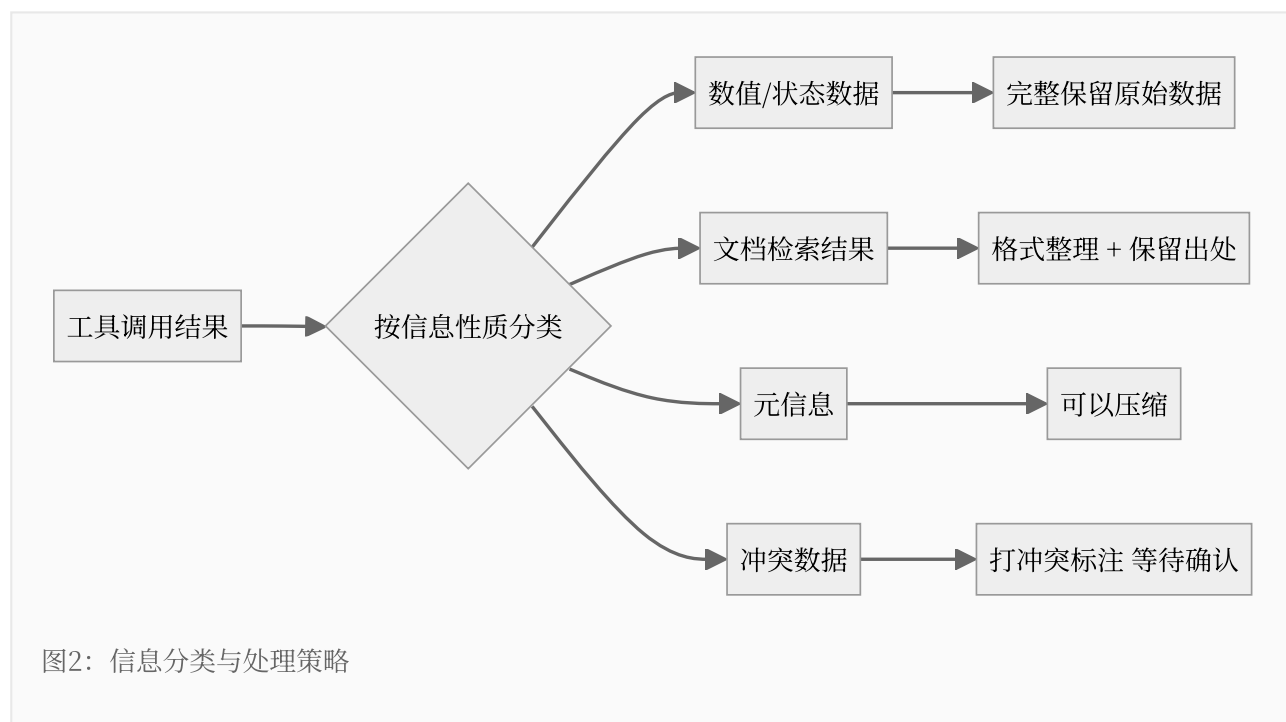
取回来的信息，怎么放进上下文？

现在工具调用完成了，Agent 手里有：

- 账务系统返回的余额明细（数值型）
- 风控系统返回的还款记录（数值 + 状态）
- 征信报告（结构化文本，几十条记录）
- 政策条款检索结果（非结构化文本，多个片段）

最简单的做法：全部放进上下文。问题是上下文窗口有限，而且这四类信息的性质完全不同——有些信息不能动，有些可以整理，有些可以压缩。如果统一处理，要么窗口很快耗尽，要么压缩了不该压缩的内容。

所以系统的第二个设计问题是：不同性质的信息，需要不同的处理策略。

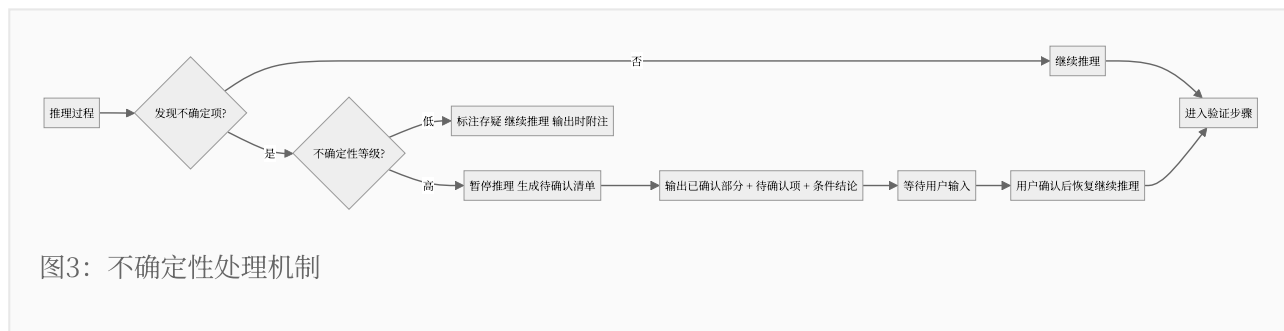


这里有一个在通用场景里比较少见的问题：**冲突数据**。如客户在系统内登记的行业分类是制造业，但工商信息里显示的是贸易业。两个数据来源，两个不同的值，而这个值直接决定哪条政策适用。通用 Agent 的做法通常是取较新的数据，或者摘要成“行业分类存在不一致”然后继续推理。但在金融场景里，这个冲突不能被 Agent 自动决策——它需要被打标、被输出、等待人工确认。这就是 F 这个分支的逻辑。

遇到不确定的信息，Agent 继续推理还是暂停

现在我们有了一条打了冲突标注的数据。系统面临一个选择：忽略冲突继续推理，还是在这个节点暂停，把问题抛给用户？这是金融 Agent 系统里较为容易忽视的一个决策点。如果继续推理，Agent 会基于不确定的前提给出一个“有说服力的错误”——答案看起来完整，但建立在一个未经确认的假设上。

所以系统的第三个设计问题是：**不确定性的处理机制**：什么时候继续，什么时候暂停，暂停之后怎么恢复。

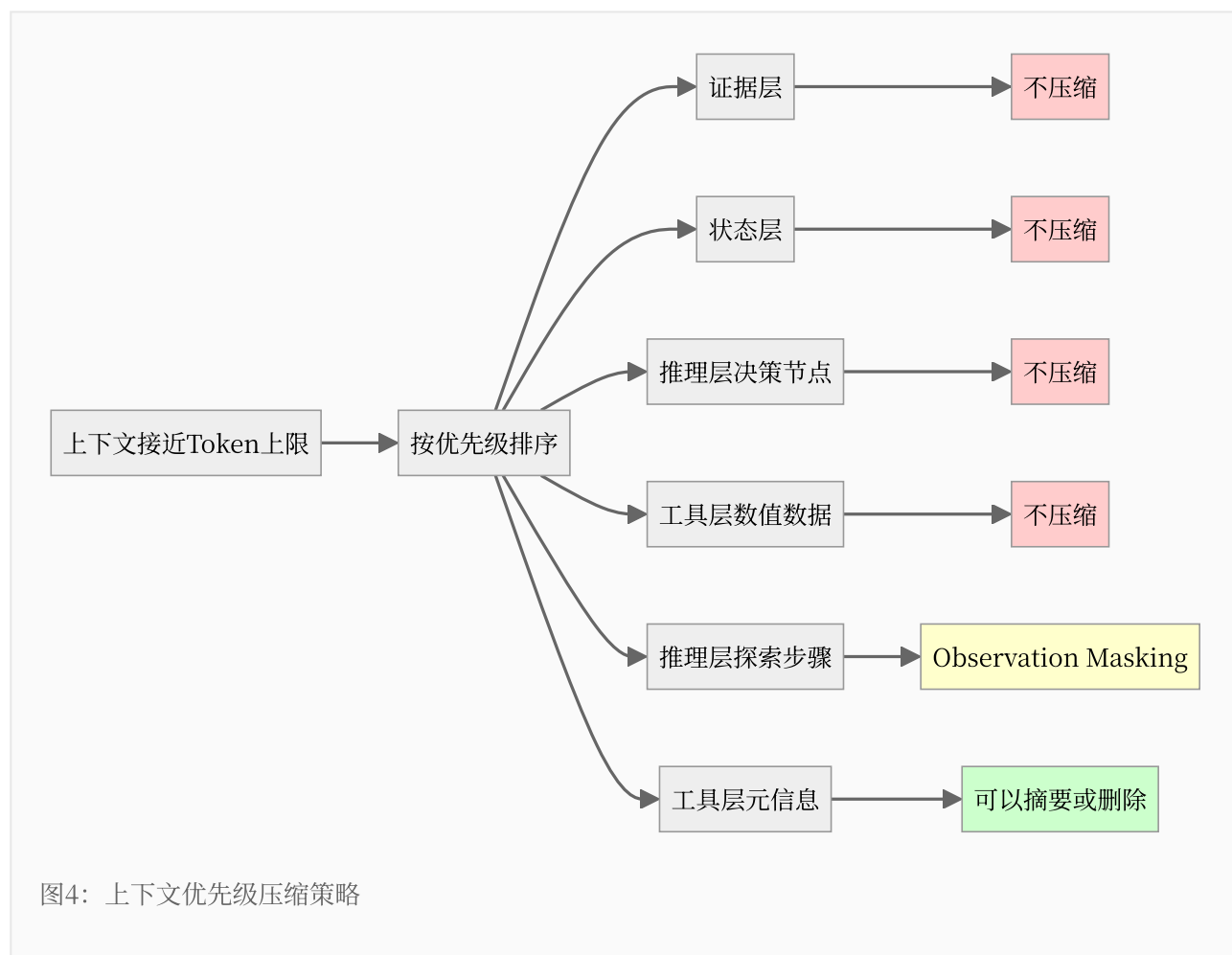


这里有一个体验工程的细节：**暂停之后要能恢复**。Agent 不是从头重新跑，而是从状态层里拿到已经确认的信息，只补充用户刚刚确认的那一条，然后从中断的节点继续。这要求状态层的设计必须支持"任务进度记录"，知道推理走到了哪一步、哪些前提已经确认、哪些还有缺失。

上下文快满了，保留什么、丢弃什么？

推理过程走了几步之后，上下文里已经有了：工具调用结果、政策条款片段、CoT 推理步骤、用户的历史输入。这时候上下文接近上限了。最简单的做法：截断最早的内容。问题是最早的内容可能是证据层的关键条款——截断了，推理结论就失去了依据，而且事后审计的时候说不清楚这个结论从哪来的。

所以系统的第四个设计问题是：上下文不够用时，按优先级而不是按时间顺序来决定保留什么。

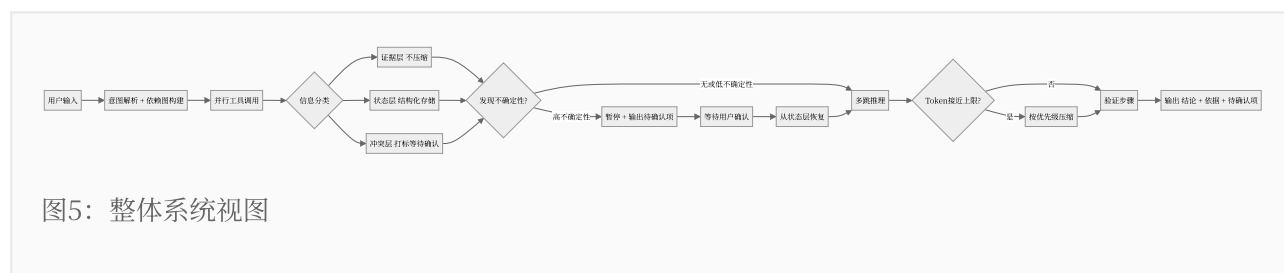


市场上的主流做法是 LLM 摘要，让模型把历史内容总结一遍，替换掉原始内容。Cursor、OpenHands 等工具都曾用这个思路。NeurIPS 2025 有一篇 JetBrains Research 的论文"The Complexity Trap"做了系统对比，发现Observation Masking——直接把旧的工具输出替换成占位符，保留推理历史——在成本上比原始 Agent 降低约 50%，效果和 LLM 摘要基本持平，有时候还略优。

推理历史保留，工具输出按优先级 Masking——这也和我们分层设计的逻辑是一致的。区别是金融场景需要额外明确哪些工具输出属于"不能被 Mask 的数值数据"。

整体系统视图

把四个设计决策放在一起，就可以看出一套agent 的雏形体系。



金融长文本的复杂性

搭建完系统后再再来看金融长文本的复杂性，金融场景里真正的挑战不是文档长，是信息结构复杂。容量问题是可以资源换的——换更大的上下文窗口，或者更高效的检索。但信息结构的问题是换不了的：跨文档的逻辑依赖、非线性的条款结构、多源头数据之间的冲突，这些问题不会因为上下文窗口变大而消失。

如何管理好上下文、信息以什么顺序进、在什么时候退出。这是一个主动管理的问题，不是容量问题。

现在回头看，前一个模块里展示的四层信息结构——证据层、状态层、推理层、工具层——看起来很整齐。但并不是一开始就存在的，大多数的时候，我们能接触到的更多是一个通用的解决方案，它覆盖了流程上的最小路径，当真正在现实业务中运转时，会发现大量的异常case。

比如：早期版本里，Agent检索到政策条款之后，会让模型先总结一遍，把几段相关的条款整合成一段话，再放进上下文。理由是这样更紧凑，节省token。结果是：当推理结论出来、我们想验证依据的时候，发现存进上下文的是“政策要求制造业客户逾期后须降级审批”，但不知道这句话来自哪份文档的第几页，也不知道原话是什么。事后追溯不了。

这个失败让我们意识到证据层的内容必须保留原文和出处，不能做任何形式的内容压缩。系统在运行的过程中不断的吸收case补充系统边界，但是仍无法保障一定可以覆盖全量的用户场景，所以如何思考系统边界与边界之外的常用应该如何应对。

边界与应用场景

最后，宏观来看一下这类系统的边界在哪。

此类技术路线的场景，有一个共同结构：答案不在一个地方，需要从多份材料里建立证据链，而且对例外条款的识别会直接影响结论。具体到金融行业，这五类场景都符合这个结构：投研分析、合规审查、客服理赔、合同审查、风控尽调。

边界在哪：凡是需要承担最终结论责任的判断，Agent做的是辅助，不是决策。授信审批需要有人签字，理赔核赔需要有人负责，合规意见需要合规人员出具。这不是因为技术不够成熟，是因为金融行业的责任结构决定的——**责任在人，不在系统。**

所以设计这类系统的时候，有一件事比技术选型更重要：**你要先想清楚，哪些输出是Agent可以直接给的，哪些必须走人工确认，哪些是在任何情况下都不应该由Agent下结论的。**把这条线划清楚，系统才真正能用。

我们需要思考的问题是如何来去构建在金融任务中的信息状态，才可以让AI在处理任务的时候可以获得足够完善的信息，并且可以做好token的管理，避免出现过度的token消耗。